PRÁCTICA 1:

DISEÑO, EXPLOTACIÓN Y OBSERVABILIDAD

FICHA TÉCNICA DE LA ACTIVIDAD

Unidad Didáctica	**UT1**: Instalación, Configuración y Gestión de Logs en MySQL 8.x
Nombre	**Práctica 1:** Diseño, explotación y observabilidad
Resultado de Aprendizaje (RA)	**RA1:** Identifica los elementos de un sistema gestor de bases de datos analizando sus funciones y relación con el sistema operativo. **RA2:** Instala el sistema gestor de bases de datos especificando las opciones de configuración y describiendo la estructura de sus elementos.
Criterios de Evaluación (CE)	**CE 1.a, CE 1.b, CE 2.a, CE 2.b, CE 2.c, CE 2.e, CE 2.f**
Tiempo Estimado	6 horas

MÉTODO DE ENTREGA Y FORMATO

- **[ ] Documento:** Una memoria detallada de la práctica en formato **PDF** (Practica_Completa_UT1_NombreAlumno.pdf). El documento debe estar justificado, con tipografía profesional, capturas de pantalla de tamaño adecuado con comentarios explicativos debajo de cada una y portadas limpias.
- **[ ] Archivos de Código:** Un archivo comprimido (Fuentes_Repaso_NombreAlumno.tar.gz) que contenga:
    1. El script SQL de inicialización completo (tienda_repaso.sql).
    2. El fichero de orquestación del escenario (docker-compose.yml).
    3. El fichero de parámetros personalizados (custom_mysqld.cnf).
- **[ ] Enlace/Texto:** Subida del archivo comprimido a través de la sección correspondiente en la plataforma virtual Moodle Centros. Esta práctica

contempla la validación de autoría mediante una posible defensa oral en clase.

1. DESCRIPCIÓN GENERAL. OBJETIVOS

En un entorno de producción, la labor de un Administrador de Sistemas (SysAdmin) no está desligada de la lógica del software de negocio. Para diagnosticar problemas de rendimiento, bloqueos en los hilos de ejecución, accesos de red sospechosos o inconsistencias en los datos, es vital comprender el diseño relacional y el comportamiento de las consultas que las aplicaciones clientes envían de forma continua al motor de bases de datos.

Esta práctica integrada tiene como objetivos:

1. **Consolidar los pilares de primer año (Módulo: Gestión de Bases de Datos):** Diseñar de forma óptima el esquema relacional, implementar la base de datos física mediante lenguaje de definición de datos (DDL) y poblar y consultar los datos aplicando lenguaje de manipulación de datos avanzado (DML).

2. **Aplicar los contenidos de administración de segundo año (Módulo: ASGBD):** Desplegar de forma aislada el gestor (MySQL 8.x) empleando contenedores con Docker, estructurar el almacenamiento persistente e interfaces de red, robustecer el servidor parametrizando variables de sistema físicas y dinámicas, configurar los archivos de log para auditoría, simular latencia e interrogar las tablas de performance_schema para diagnóstico forense.

2. CONSIDERACIONES PREVIAS

- **Escenario Tecnológico:** Se empleará obligatoriamente **Docker** y **Docker Compose** en entornos Linux para garantizar la uniformidad en el aula y aislar las variables del sistema host local.

- `Control Estricto de Copias: Para validar el desarrollo, todas las capturas de la terminal del host deben mostrar claramente el nombre de usuario de la máquina del alumno (ej: alumno_asir@pc-laboratorio:~$). Para las capturas de consultas SQL dentro de la terminal de MySQL, el alumno deberá ejecutar como paso previo la siguiente consulta de validación:`

```
SELECT 'Desarrollado por: <Nombre_Alumno> - IP: <IP_Aula>'
AS autor, NOW();
```

3. ACTIVIDADES A REALIZAR

La práctica de repaso y aplicación se desarrollará siguiendo estrictamente las siguientes cuatro fases.

FASE 1: MODELADO RELACIONAL E IMPLEMENTACIÓN DDL [Repaso 1º GBD]

A partir del problema clásico del comercio electrónico, debes realizar el paso al modelo relacional e implementar físicamente la estructura de datos.

Lógica de Negocio y Reglas de Integridad:

- **usuario:** Almacena nif (PK), correo (único y obligatorio), nombre, direccion, telefono, login (único y obligatorio) y password (obligatorio, guardado en formato cifrado).
- **seccion:** Almacena un codigo_seccion (entero autoincremental, PK), nombre (único y obligatorio), descripcion y fecha_creacion (fecha del sistema por defecto).
- **producto:** Almacena codigo_producto (cadena de caracteres de longitud variable, PK), nombre (obligatorio), descripcion, precio (decimal con dos decimales para los céntimos, obligatorio) y el codigo_seccion al que pertenece de forma obligatoria (FK).
- **carrito:** Almacena codigo_carrito (autoincremental, PK), fecha_compra (tipo fecha, obligatorio), completado (booleano que indica si la compra se cerró o sigue activa) y el nif del usuario dueño del carro (FK).
- **detalle_carrito:** Relación N:M que asocia un carrito con múltiples productos, indicando la cantidad de unidades solicitadas de cada producto (entero positivo). La clave primaria de esta tabla intermedia es compuesta por la unión de las dos claves ajenas.
- **datos_bancarios:** Almacena el numero_tarjeta (cadena de longitud estándar de tarjeta, PK), fecha_caducidad (tipo fecha, obligatoria), cvv (entero de 3 dígitos, obligatorio) y el nif del usuario asociado (FK).

Tareas técnicas a realizar:

1. Diseña el esquema relacional formalizando claves primarias, claves ajenas y restricciones de nulidad y unicidad.
2. Escribe el script SQL completo denominado tienda_repaso.sql. La base de datos debe utilizar obligatoriamente el motor transaccional **InnoDB** y el juego de caracteres **utf8mb4**.
3. `Configura de forma explícita las claves foráneas para que los flujos de borrado y modificación se propaguen automáticamente en cascada (ON DELETE CASCADE y ON UPDATE CASCADE) en las relaciones críticas entre usuarios, carritos, secciones y productos.`

FASE 2: ORQUESTACIÓN DE RED Y ALMACENAMIENTO BAJO DOCKER [ASGBD]

Desplegarás el servidor MySQL utilizando contenedores para asegurar la portabilidad y aislar los logs del sistema operativo host.

Tareas técnicas a realizar:

1. Crea un directorio de trabajo local denominado repaso_ut1/. Dentro de él, genera los directorios vacíos ./config y ./logs.
2. Diseña un archivo docker-compose.yml que levante una instancia de MySQL 8.x con:
    - Nombre del contenedor: mysql-sysadmin-repaso.
    - El puerto 3306 mapeado de manera unificada al exterior.
    - Volumen de persistencia para el directorio de datos internos del motor (/var/lib/mysql).
    - Un montaje tipo *Bind-Mount* que asocie ./config con la ruta de carga de ficheros de configuración modular de MySQL (/etc/mysql/conf.d).
    - Un montaje tipo *Bind-Mount* que asocie ./logs con el directorio interno de escritura de logs del contenedor /var/log/mysql.
3. Crea un archivo ./config/custom_mysqld.cnf e introduce las siguientes directivas fijas:
    - `Desactivar confirmación automática (autocommit = OFF).`
    - Fijar expiración de contraseñas a los 180 días (default_password_lifetime = 180).
    - Habilitar un límite de conexiones simultáneas de 120.
4. Levanta el contenedor de Docker (docker compose up -d) y, mediante consultas SQL a las variables de entorno, verifica que el motor ha adoptado las configuraciones estáticas definidas en el fichero de configuración de la carpeta local ./config.

FASE 3: CARGA DML, PERSISTENCIA EN CALIENTE Y EXPLOTACIÓN DE DATOS [Integrado 1º/2º]

Poblarás tu base de datos y simularás el comportamiento real de un cliente interactuando con la tienda virtual de comercio electrónico, verificando el registro de logs del sistema.

Tareas técnicas a realizar:

1. `Añade sentencias INSERT en tu script SQL para cargar la información de prueba:`

    - 3 usuarios registrados (uno de ellos debe tener el login mi_usuario y su contraseña debe guardarse de forma cifrada mediante una función de hash como SHA2).
    - 3 secciones en la tienda (una con el nombre exacto Mi_sección y otra llamada Electrónica).
    - 5 productos asignados a las diferentes secciones creadas.
    - 2 carritos virtuales (un carro completado y otro pendiente de finalizar compra para el usuario mi_usuario).
    - 1 registro de datos bancarios asociado al usuario mi_usuario.

2. Modifica el servidor en caliente de forma **persistente y global** utilizando sentencias directas SQL (SET PERSIST) para los siguientes requisitos de observabilidad:

    - `Activar el General Query Log enviando la salida a /var/log/mysql/query.log.`
    - `Activar el Slow Query Log con un umbral de tiempo máximo de long_query_time = 1.500000, enviando la salida a /var/log/mysql/slow.log.`
    - `Activar el registro de consultas ineficientes que no utilizan índices (log_queries_not_using_indexes).`
    - Comprueba en tu terminal del host que se ha generado de forma automática el archivo JSON mysqld-auto.cnf en el volumen de datos de tu contenedor de base de datos.

3. Resuelve y ejecuta las siguientes consultas de negocio sobre tu esquema, asegurando que se capturen en tu log general:
    - **Consulta de login:** Obtener el password cifrado del usuario con login mi_usuario para verificar accesos correctos.
    - **Secciones del mes:** Mostrar las secciones que se han creado este mes actual.
    - **Coste del carro:** Visualizar el coste total de un carrito de compra concreto, calculando el sumatorio de la cantidad por el precio de cada producto, filtrando para el usuario con login mi_usuario y fecha de compra 12/09/2023.
    - **Verificación de flujos en cascada:** Ejecuta la sentencia SQL para borrar un producto de la sección Mi_sección. Documenta qué ocurre con los registros de la tabla intermedia detalle_carrito y explica cómo ha operado el motor InnoDB.

FASE 4: OBSERVABILIDAD, LATENCIA Y AUDITORÍA INTERNA [ASGBD]

Diagnosticarás problemas de latencia provocados en el sistema y auditarás el estado de la instancia consultando las tablas del catálogo.

Tareas de verificación a realizar:

1. Provoca una consulta lenta de alta latencia artificial (3 segundos) mediante la función SLEEP().
2. Inspecciona desde la terminal de tu sistema operativo host el archivo local ./logs/slow.log y aporta una captura donde demuestres su registro. Interpreta técnicamente cada una de las métricas que acompañan al registro de la consulta en el archivo: Query_time, Lock_time, Rows_sent y Rows_examined.
3. Fuerza un error de sintaxis de forma intencionada e identifica su código numérico devuelto por MySQL. Escribe una consulta SQL sobre la tabla performance_schema.events_errors_summary_by_user_by_error que muestre los últimos errores ocurridos para tu usuario administrador, comprobando que se ha registrado tu código de error provocado.
4. `Escribe y ejecuta una consulta SQL sobre la tabla performance_schema.status_by_user que permita auditar la cantidad total de sentencias DML de tipo SELECT realizadas por el usuario administrador de la base de datos.`

4. CALIFICACIÓN

> ⚠️ **IMPORTANTE:** El retraso en el plazo de entrega a través de la plataforma, la entrega del documento en formatos editables no permitidos (como .docx), capturas con identificación del alumno/a o la omisión de los enlaces de las fuentes documentales utilizadas penalizará gravemente la nota o invalidará la práctica.

La actividad será evaluada según los siguientes criterios de ponderación base: Funcionalidad (50%), Memoria y Documentación (40%), Presentación (10%).

CRITERIOS DE CALIFICACIÓN (RÚBRICA GENERAL)

Apartado	Excelente (100%)	Aceptable (60%)	Insuficiente (0%)	Peso
Fase 1: Diseño y Script DDL (1º)	Diseño relacional correcto. Script SQL libre de errores. Implementa restricciones Unique y flujos de borrado/modificación cascade de manera idéntica a los requisitos de diseño del problema.	El script presenta pequeños fallos sintácticos. Faltan flujos cascade en claves ajenas críticas o restricciones Unique.	No realiza el diseño relacional, no entrega el script SQL o las tablas no son transaccionales en InnoDB.	**20%**
Fase 2 y 3: Docker, Carga y Logs (2º)	Docker Compose funcional con volumen persistente y montajes bind. Archivo custom_mysqld.cnf correcto. Carga de datos DML consistente con password cifrado. Configura de forma persistente (SET PERSIST) los logs.	Contenedor funcional pero montajes bind erróneos. Faltan variables por parametrizar en el custom.cnf o la carga de datos es incompleta.	No despliega con contenedores, no activa los logs en caliente o no realiza la carga de datos.	**50%**
Fase 4: Diagnóstico y performance_schema	Interpreta impecablemente las métricas del Slow Query Log. Realiza con total soltura consultas estructuradas de auditoría sobre las tablas	Consulta las tablas de rendimiento pero presenta dificultades para asociar el error provocado	No realiza consultas sobre las tablas de performance_schema o no documenta el apartado.	**30%**

Apartado	Excelente (100%)	Aceptable (60%)	Insuficiente (0%)	Peso
	internas de performance_schema.	con el código numérico.		

CRITERIO ESPECÍFICO DE DOCUMENTACIÓN Y CAPTURAS (1.5 Puntos)

Criterio	Excelente (1.5 Puntos)	Aceptable (0.75 Puntos)	Insuficiente (0 Puntos)
Calidad de la documentación y validación	Memoria impecable en PDF. Las capturas son legibles, se identifica claramente al autor en el prompt de la consola o mediante consulta SQL explicativa y se detalla técnicamente cada fase.	Documento bien estructurado pero faltan comentarios explicativos en alguna captura de pantalla o la validación del alumno no es clara en todas las imágenes.	El documento es desordenado, de difícil lectura, sin capturas justificadas, o se sospecha copia entre alumnos.